

Anthony Sesay
5/12/26
CMSC 461

Final Report For Pluto Realty Database

Phase A:

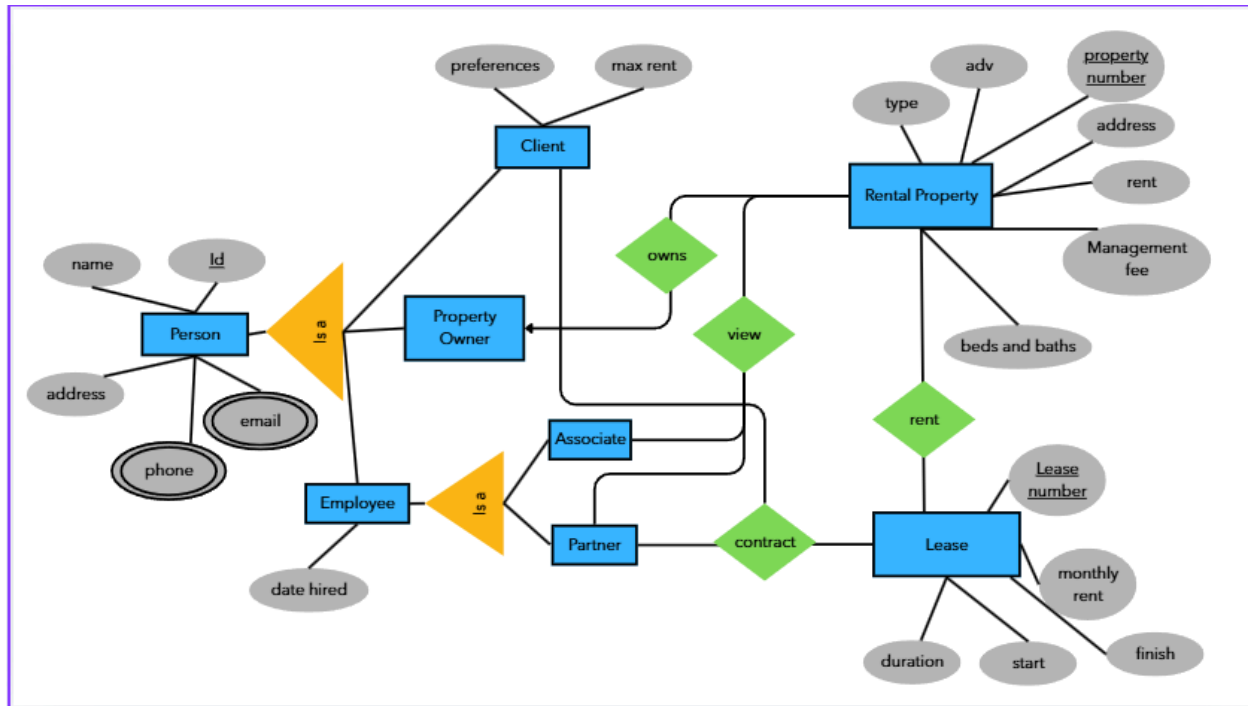
I am tasked to design The Pluto Realty database, which is an application intended to support the operations of Pluto Realty, Inc. by allowing employees to manage rental properties, clients, owners, viewings, and lease contracts through a centralized system. The system will be accessed by employees through web-enabled computers. The requirements describe several core entities: employees, property owners, clients, rental properties, property viewings, and leases.

Employees are divided into two roles: partners and associates, which affects their responsibilities in the system. Associates manage properties and conduct property viewings for clients, while partners oversee property owners and are responsible for creating lease agreements. Each property has attributes such as location, property type, size, rent, and management fee, and must be linked to exactly one owner and one associate. Owners and clients store similar personal information: name, address, phone numbers, and emails. However, owners are overseen by partners while clients include preferences and budget information. Additional constraints exist in the requirements, such as associates managing at most five properties, lease durations being between three and thirty-six months, and lease numbers being unique per property.

From a database design perspective, the system will require identifying the main entities and their relationships and converting them into a relational schema. The entities such as Employee, Property, Owner, Client, Viewing, and Lease will be designed into tables, while relationships, such as owning a property will be implemented through foreign keys. Some attributes are multivalued, such as phone numbers and email addresses. At the end of the project, once the database has been created and populated, the project requires implementing a function to calculate management fees over the past three months and that automatically updates the property advertisement flag when a lease is created.

Several files that create, populate and clear the database are needed. These files should be separate and be able to run specific queries. Lastly, a client application (Jupyter Notebook) designed to provide a user-friendly front end for interacting with the Pluto Realty database system is required. The application includes input cells that accept user queries or parameters, such as property identifiers, client details, or search criteria, and output cells that display query results or system feedback in a structured and readable format.

Phase B:



In this Er diagram that designs a database system for the Pluto Real estate system:

There are the entities:

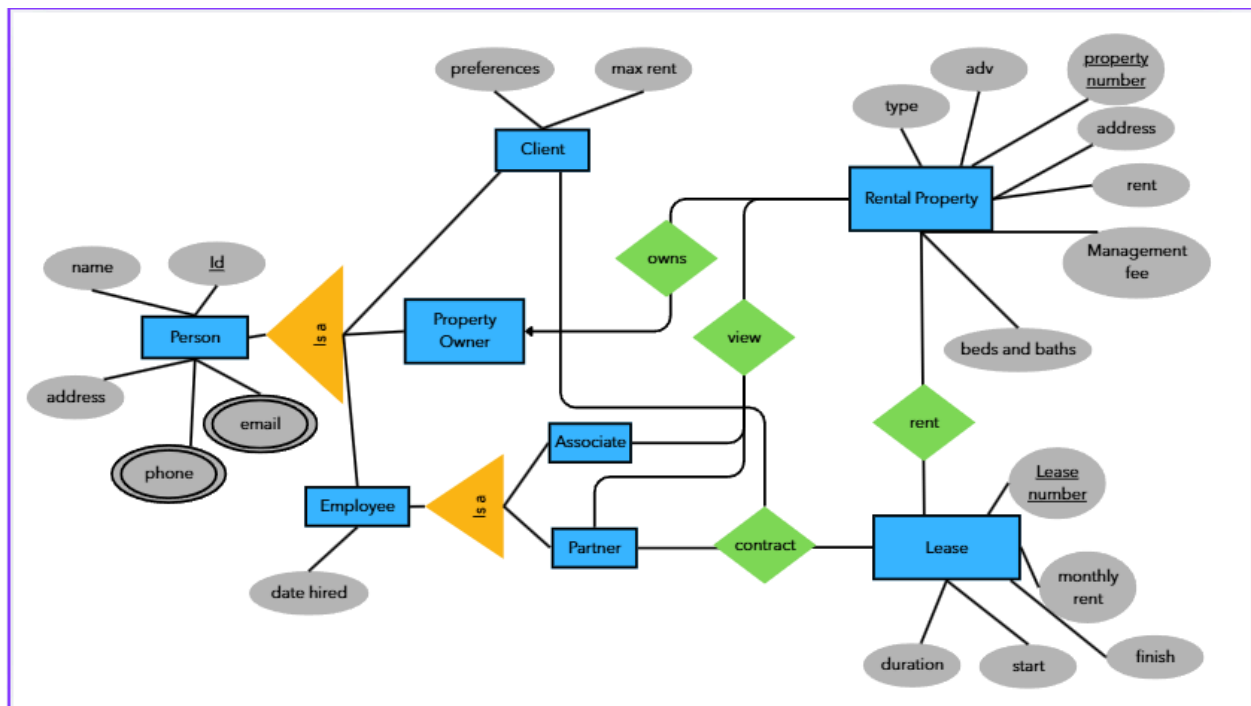
- Person (id, name, address, phone, email)
 - This entity is a generalization of all people within the schema. It contains the attributes: id, name, address, phone, and email. The id is the primary key used to identify all people
- Employee (employee id, hired date)
 - Specialization of person. This add as the date, hired attribute and the employee number that would be used to identify it is taken from the id in the person entity
- Property owner
 - Specialization of person entity. Has relations specific to this kind of entity
- Client (property preferences, max_rent)
 - Specialization of Person. Adds the property preference and max rent attributes. Has its own specific type of relations compared to a person
- Partner (is_partner)
 - Specialization of employee that has the ability to lease a contract unlike an associate
- Associate (is_associate)
 - Specialization of employee
- Rental Property
 - Property number is the primary key
 - Only one owner per property
- Lease (lease number, rent, deposit, duration, start, finish)
 - Entity that has the lease number as its primary key.

Relations

- Lease a contract
 - Relationship between partners, clients, and lease which allows a lease to be created so that a client can rent a property
- view (associate, client, date)
- Own a property
 - Relates Property owners and properties to allow a reference to properties from property owners
- Rent
 - Allows renting to occur of a lease contract has been formed

Phase C:

For this phase, the conceptual design for the database from Part B is modeled by this ER diagram:



To map this design into a relational data model, a series of reductions on each of the entities and relations were performed to change this ER model to a relational schema.

- Person (id, name, address)
 - Pk: id
 - Fk: none

This is the main entity representing all people in the database. It groups common attributes shared across all roles among people to avoiding redundancy

- Phone (id, phone)
 - PK: (id, phone)
 - FK: id → Person(id)
- Email (id, email)
 - PK: (id, email)
 - FK: id → Person(id)

Phone and email are multi-valued attributes and this allows a person to have multiple phone numbers and emails

- Employee (employee id, hired_date)
 - Pk: employee id
 - Fk: employee_id → Person(id)

Specialization of person that adds the hired_date attribute. This is a separate table where an employee inheriting their values in person is implemented via foreign key reference

- Property owner (id)
 - Pk: id
 - Fk: id → Person(id)

Specialization of person entity. Has relations specific to this kind of entity so this was made into its own table and is linked to the person table through a foreign key reference

- Client (id, property preferences, max_rent)
 - Pk: id
 - Fk: id → Person(id)

Specialization of Person. Adds the property preference and max rent attributes. Has its own specific type of relations compared to a regular person

- Partner (employee_id)
 - Pk: employee id
 - Fk: employee_id → Employee(employee_id)

Specialization of an employee that has the ability to lease a contract unlike an associate. It is separated to enforce role-specific constraints and relationships

- Associate(employee_id)
 - Pk: employee_id
 - Fk: employee_id → Employee(employee_id)

Specialization of employees. Relation created for an Employee representing employees responsible for assisting clients

- Rental Property(property_number, type, address, rent, management_fee, bed_baths, adv, associate)
 - Pk: property_number
 - Fk: associate → Associate(employee_id)

This entity represents properties available for rent. This relation is its own base entity.

- Lease (lease_number, property_number, rent, deposit, duration, start, finish)
 - Pk: lease_number, property_number
 - Fk: property_number → RentalProperty(property_number)

This lease relation identifies a lease tied to a specific property. A composite primary key is used to ensure that a lease is uniquely identified within the context of a property

- Contract(partner_id, lease_number, client_id, property_number)
 - Pk: partner_id, lease_number, property_number, client_id
 - Fk: partner_id → partner(employee_id)
 - client_id → client(id)
 - property_number → Rental Property(property_number)

This relation models the ternary relationship between Partner, Client, and Lease. It captures which partner arranged a lease for which client and property.

- View(client_id, associate_id, property_number, view_time)
 - Pk: client_id, associate_id, property_number
 - Fk: client_id → Client(id)
 - associate_id → Associate(employee_id)

- `property_number` → `RentalProperty(property_number)`
- `Owns(owner_id, property_number)`
 - PK: `owner_id, property_number`
 - FK: `owner_id` → `PropertyOwner(id)`
 - `property_number` → `RentalProperty(property_number)`

The rent relation is excluded from this relation as it would be represented as the identifying keys of the Rental Property and Lease entities, however this is already captured by the Lease entity as its primary key is (`lease_number`, `property_number`)

Phase D:

For this phase, I converted the relations from Phase C into SQL tables in order to physically create my conceptual database. The goal was to translate each relation above into a sql statement to create a table while enforcing integrity constraints such as primary keys, foreign keys, and domain restrictions.

Phase E:

In this phase, a Jupyter Notebook was developed to serve as a user-facing interface for interacting with the Pluto Realty database. The notebook connects to the MySQL database using SQL magic commands and allows users to execute queries directly within a Python environment. The notebook is intended to work inside of VS code or JupyterLab with MySQL server. The notebook supports insertion and deletion of tuples, viewing data via select queries, and enforcing constraints with triggers and transactions.

Phase F:

In this final phase, a user guide was integrated directly into the Jupyter Notebook to support usability and ensure that employees can interact with the system without requiring database expertise. It explains how to use and run sql statements and how to use the database given the files.